

VerilogDB: The Largest, Highest-Quality Dataset with a Preprocessing Framework for LLM-based RTL Generation

PAUL E. CALZADA, ZAHIN IBNAT, TANVIR RAHMAN, KAMAL KANDULA, DANYU LU, SUJAN KUMAR SAHA, FARIMAH FARAHMANDI, and MARK TEHRANIPOOR, University of Florida, USA

Large Language Models (LLMs) are gaining popularity for hardware design automation, particularly through Register Transfer Level (RTL) code generation. In this work, we examine the current literature on RTL generation using LLMs and identify key requirements for training and fine-tuning datasets. We construct a robust Verilog dataset through an automated three-pronged process involving database (DB) creation and management with PostgreSQL [14], data collection from code hosting sites like OpenCores and GitHub, and data preprocessing to verify the codes' syntax, run logic synthesis, and extract relevant module metadata. We implement a scalable and efficient DB infrastructure to support analysis and detail our preprocessing pipeline to enforce high-quality data before DB insertion. The resulting dataset comprises 20,392 Verilog samples, 751 MB of Verilog code data, which is the largest high-quality Verilog dataset for LLM fine-tuning to our knowledge. We further evaluate the dataset, address associated challenges, and explore potential applications for future research and development in LLM-based hardware generation.

CCS Concepts: • **Hardware** → **Hardware description languages and compilation**; • **Computing methodologies** → **Neural networks**; • **Information systems** → **Data cleaning**.

Additional Key Words and Phrases: verilog, large language models, EDA, RTL generation, machine learning for hardware design, dataset.

ACM Reference Format:

Paul E. Calzada, Zahin Ibnat, Tanvir Rahman, Kamal Kandula, Danyu Lu, Sujan Kumar Saha, Farimah Farahmandi, and Mark Tehranipoor. 2025. VerilogDB: The Largest, Highest-Quality Dataset with a Preprocessing Framework for LLM-based RTL Generation. 1, 1 (July 2025), 28 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

The push for quicker time-to-market (TTM) in the semiconductor industry has heightened the necessity for automation in hardware design, aiming to enhance power, performance, and area (PPA) metrics. This automation of hardware design, particularly for System-on-Chip (SoC) development, is rapidly evolving with the emergence of LLMs. The generative artificial intelligence (GenAI) models, initially developed for natural language and high-level programming tasks, are increasingly being adapted to support code generation in hardware description languages (HDLs) such as Verilog. However, for LLMs to be effective in this domain, they require clean, diverse, and well-structured training datasets that reflect real-world RTL design patterns and constraints.

Authors' Contact Information: [Paul E. Calzada](mailto:paul.calzada@ufl.edu), paul.calzada@ufl.edu; [Zahin Ibnat](mailto:ibnatz16@ufl.edu), ibnatz16@ufl.edu; [Tanvir Rahman](mailto:tanvir.rahman@ufl.edu), tanvir.rahman@ufl.edu; [Kamal Kandula](mailto:kamalkandula@ufl.edu), kamalkandula@ufl.edu; [Danyu Lu](mailto:danyulu@ufl.edu), danyulu@ufl.edu; [Sujan Kumar Saha](mailto:sujansaha@ufl.edu), sujansaha@ufl.edu; [Farimah Farahmandi](mailto:farimah@ece.ufl.edu), farimah@ece.ufl.edu; [Mark Tehranipoor](mailto:tehranipoor@ece.ufl.edu), tehranipoor@ece.ufl.edu, University of Florida, Gainesville, Florida, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

Recent efforts such as *VeriGen* [44] and *VerilogEval* [21] have highlighted both the promise and limitations of applying LLMs to Verilog generation. These studies demonstrate that while LLMs can produce syntactically valid Verilog, their ability to generalize and generate semantically meaningful logic is heavily dependent on the quality and diversity of the training data. For instance, *VeriGen* fine-tuned a code generation model using GitHub-sourced Verilog code and textbook examples, but the dataset required extensive manual filtering to remove simulation-specific or non-synthesizable files. Similarly, *VerilogEval* established a standardized evaluation suite for Verilog LLMs but noted performance degradation in modules that deviated from the model’s perceived patterns.

The absence of high-quality curated Verilog datasets remains a significant area for improvement. Verilog code available in public repositories is not particularly reliable when used for data-driven research [21]. While Verilog is a standardized HDL, the code found in open-source platforms such as GitHub or OpenCores varies significantly in quality, style, and completeness. Projects often differ in indentation, naming conventions, commenting practices, and overall formatting. Some modules are clearly defined and self-contained, while others are deeply nested within larger designs or split across multiple files with ambiguous dependencies. These inconsistencies cause problems both in manual inspection and in automated parsing [21]. Moreover, Verilog frequently includes preprocessing directives, macros, and vendor-specific syntax, which cannot be used for standardization.

For LLMs to effectively generate or understand Verilog code, high-quality and consistently structured training data is needed. Unlike high-level programming languages, where syntactic errors may be caught and corrected during runtime or compilation, errors in HDL can propagate through synthesis and further to the bitstream that will ultimately cause major problems in hardware. These errors can exist in major sources of data and as a result, LLMs trained on poorly curated HDL corpora are prone to generating incorrect or logically incoherent outputs, which is not only unhelpful but potentially dangerous in security-critical systems [47]. This proves the need to increase dataset quality in LLM-driven hardware design generation.

To address the aforementioned issues, we develop *VerilogDB*: a scalable and automated pipeline to collect, sanitize, and structure Verilog RTL data from diverse open-source locations. Such a DB allows for the storage and indexing of HDL samples in a way that supports scalability, filtering, and metadata tagging [2]. Our primary objective is to produce a high-quality dataset suitable for LLM fine-tuning in the context of IP and SoC design generation. The contributions of this work are as follows:

- We curate a large corpus of Verilog RTL files by scraping GitHub, OpenCores, and university course materials, while applying structured keyword filters to exclude simulation-only or library-based repositories. We used a robust filtering approach to exclude codes with non-synthesizable constructs, testbenches, and system-verilog codes. We are the first, to our knowledge, to include hierarchical designs in the dataset.
- We implement an extensive preprocessing pipeline that includes duplicate removal (using content-based hashing), syntax validation (using *iverilog* [48]), and logic synthesis checking (using *Yosys* [49]). This flow is automated compared to other works that rely on manual sample preparation and validation. We are the first, to our knowledge, to include explicit logic synthesis checks in Verilog data cleaning.
- From the cleaned modules, we extract detailed metadata including module names, I/O definitions, inline comments, token counts, and short natural language descriptions into a JSON format to support instruction-based LLM training.
- We demonstrate the scalability of our pipeline, which processes over 30 GB of raw data and produces a clean, structured dataset consisting of 20,392 synthesizable Verilog modules, at a 751 MB data size, ready for downstream

learning tasks. This dataset is the largest of its kind to our knowledge. We also analyze the dataset’s module complexity and diversity and provide further insights of our work to LLM training for RTL generation.

Section 2 discusses RTL-generation through LLMs and considerations for generated Verilog code, and explains the current state-of-the-art in LLM-based RTL generators. Section 3 presents each component of our methodology including data collection, preprocessing, and insertion into the DB. The next section provides an analysis of the resulting dataset while outlining its application to support LLM-based RTL generation and hardware design automation. The benefits and challenges of *VerilogDB* are also discussed. Section 5 concludes the work.

2 Background

2.1 LLM Fine-Tuning for RTL Generation

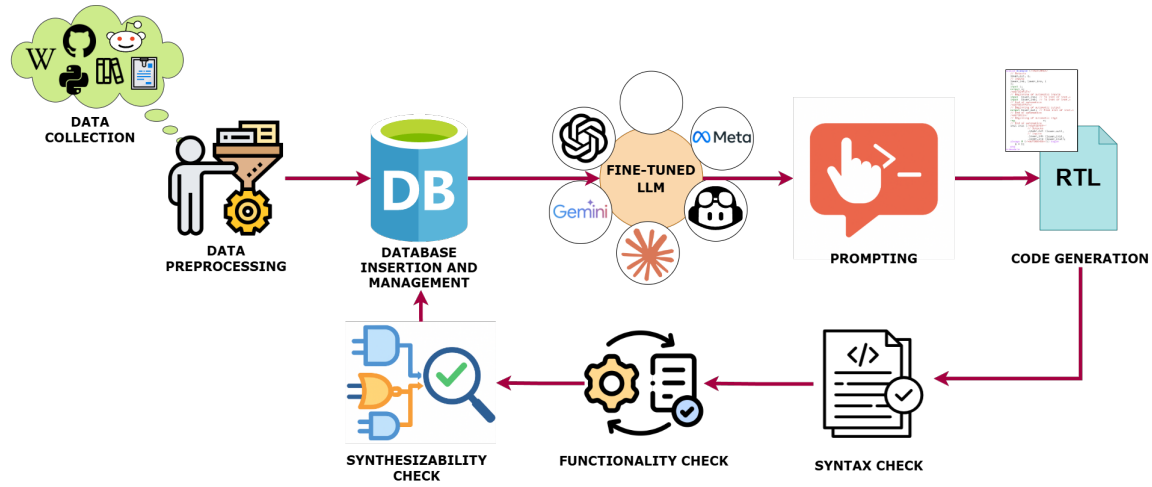


Fig. 1. High-level flow for LLM fine-tuning process including iterative feedback for closed-loop fine-tuning.

Curated domain-specific datasets have served as the backbone for many innovative ML tasks from fine-tuning to retrieval augmented generation (RAG) pipelines. As shown in Figure 1, developing a fine-tuned LLM for RTL generation must follow a meticulous workflow to optimize performance. The first step is the generation of a catered dataset of RTL sourced from diverse codebases which then undergoes rigorous preprocessing and is organized into a scalable database architecture. The sources and quality of data dictate the efficacy of LLM RTL generation, which is explored in this section with discussions on the quality of code in Section 2.2 and the subsequent results of leading LLM RTL code generation solutions in Section 2.3. Next, the stored data is used to fine-tune an open-source LLM, especially those already trained on a large corpus of code. Then, a procedural prompting approach can be deployed to generate an RTL code output set. These outputs are then evaluated by syntax, functionality via testbenches, and logic synthesis to gauge the efficacy of the model. The cleaned data is fed back into the database to further enhance the DB and overall model performance [50]. The feedback data must also be preprocessed similarly to the initial data to enforce consistency. This meticulous process is not always followed through for a large corpus of data, and as a result, can cause the fine-tuned models or agents to generate inconsistent results.

2.2 Considerations for LLM-Generated Verilog Code

The application of LLMs to hardware design tasks is a growing field, driven by the success of software code generation. While general-purpose code generation models like OpenAI’s Codex [28] and Code LLaMA [34] show strong performances on objected-oriented programming languages (e.g., Python, C++), their capabilities in the hardware domain, particularly for Verilog RTL generation, remain underexplored and are likely limited because of the scarcity of hardware-specific training data. Moreover, hardware design has strict correctness constraints. Additionally, Verilog is a concurrent, timing-sensitive language where errors may not present themselves just in syntax and simulation. Verilog also includes both behavioral and structural modeling styles, vendor-specific extensions, and preprocessor macros. As a result, domain-specific fine-tuning is crucial for LLMs to understand RTL constructs, timing, and logic synthesis requirements. The dataset must consist of:

- **Syntactically valid and synthesizable Verilog modules.**

Good:

```
module adder(input [7:0] a, b, output [7:0] sum);
  assign sum = a + b;
endmodule
```

Bad:

```
module adder(input logic [7:0] a, b, output logic [7:0] sum)
  sum = a + b
endmodule
```

The latter example includes SystemVerilog constructs in a .v file and has syntax errors.

- **Diverse structural designs**, spanning arithmetic cores, crypto engines, DSP blocks, and more.

Good:

A balanced mix of simple counters, medium-complexity crypto cores (e.g., AES), and multi-module DSP blocks.

Bad:

Lack of representative examples of the diverse module types, or contains unsynthesizable testbenches.

- **Reasonable formatting and indentation**, enabling the model to learn standardized yet diverse code structure.

Good:

2- or 4-space indents, consistent naming (e.g., `clk`, `rst`, `data_in`), and line spacing.

Bad:

```
module X(input a,output b);assign b= a;endmodule
```

- **Contextual metadata**, including I/O definitions, module names, and design comments.

Good:

```
// 8-bit up-counter with asynchronous reset
module counter(input clk, input rst, output reg [7:0] count);
```

Bad:

No comments, or only boilerplate license text. I/O is ambiguous or undocumented.

- **Concise natural language descriptions**, which serve as prompts during instruction tuning.

Good:

“This module performs an 8-bit addition between two inputs and outputs the result.”

Bad:

“This file was created by XYZ. For internal use only.” or overly verbose documentation.

In our work, we created a structured pipeline that reflects these findings. Data is obtained from trusted open-source repositories, cleaned using filtering and de-duplication tools, and validated via syntax and synthesis checks. We additionally extract metadata and descriptions for every module, allowing for prompt-response formatting compatible with instruction-tuned LLMs. The result is a dataset created specifically to support LLM training for RTL generation that is capable of generalizing across a range of module types and design idioms encountered in modern SoC development.

2.3 Prior Work

Research has shown LLMs are being applied to electronic design automation (EDA) [3, 17, 53], design verification [32, 55], hardware security [35–37], and HDL code generation [8, 11]. Early foundational studies including *ChipGPT* [8], *Chip-Chat* [7], *AutoChip* [46], and *GPT4AIGChip* [11] aimed to improve hardware design outputs produced by LLMs with prompt engineering. In recent research, there has been a trend towards domain-specific pretraining and fine-tuning, exemplified by works such as *MG-Verilog* [54], *RTLCode* [24], *Verigen* [45], and *VerilogEval* [22]. The latest efforts have sophisticated the research of fine-tuning and agentic approaches with large DBs and better frameworks. These include but are not limited to OriGen [9], BetterV [31], AutoVCoder [12], CodeV [56], CraftRTL [23], and RTL++ [1].

2.3.1 Prompt Engineering Approaches. Various techniques enhance an LLM’s RTL generation accuracy through prompt engineering. *ChipGPT* [8] improves Verilog generation outputs from widely-used LLMs (e.g. ChatGPT) through a structured prompting scheme tailored for LLMs, enhancing code quality by incorporating hardware-specific context (e.g., input-output ports, module functions). It also cleaned the output codes, correcting and optimizing generated RTL, searching the design space and selecting optimal designs based on PPA metrics. While effective in elevating generation from general-purpose LLMs, *ChipGPT* relies solely on prompt engineering without fine-tuning, and lacks support for hierarchical or multi-module designs, limiting its applicability to complex systems. *Chip-Chat* [7] utilizes a pre-trained LLM (like ChatGPT) without fine-tuning, and instead, depends on prompt engineering to steer code generation. The system incorporates a feedback mechanism in which user inputs enhance the outputs of the LLM. While it improves usability, *Chip-Chat* does not incorporate fine-tuning or domain-specific training. As a result, the generated code often requires manual correction and may not consistently meet functional or structural correctness for complex or integrated designs. *GPT4AIGChip* [11] utilizes tailored prompts to direct GPT-4, embedding hardware constraints and AI accelerator design concepts, such as data flow architectures and tiling strategies. It implements LLM-based parameter optimization (e.g., dataflow, bit-width) to enhance PPA metrics, while incorporating feedback from EDA tools to improve the generated designs. While this approach integrates architectural constraints and AI chip design knowledge, it lacks fine-tuning and depends entirely on handcrafted prompts. *RTL++* [1] proposes a graph-enhanced generation method using control and dataflow graph embeddings to strengthen semantic understanding in Verilog synthesis. The model

improves over token-based LLMs in capturing structural dependencies. However, it is primarily benchmarked on simplified designs, and the effectiveness of graph-guided generation in full-chip RTL is largely untested.

2.3.2 Fine-tuned Models. General-purpose models can be further enhanced by training them on domain-specific datasets. *MG-Verilog* [54] introduces a multi-grained dataset that addresses the limitations of existing corpora by providing Verilog code samples at varying levels of abstraction, which allows for more effective fine-tuning and in-context learning for LLMs. This dataset emphasizes the importance of diverse and well-annotated data in enhancing model performance. *MG-Verilog* also tests this dataset by fine-tuning the CodeLLaMA-7B-Instruct model. However, *MG-Verilog* has some limitations that may hinder its broader applicability. Despite containing over 11,000 samples, the dataset size remains relatively small for large-scale LLM training, potentially constraining generalization across different hardware design tasks. *RTLCoder* [24] is an RTL code generation framework that addresses the challenges of privacy concerns and the lack of high-performance open-source LLMs for Verilog generation by using a specialized training dataset and a customized model architecture. However, *RTLCoder*'s scope is constrained by the model's compact size. While it is efficient for deployment, the size limits its capacity to capture long-range dependencies or complex hierarchical design patterns often seen in industry-specific hardware designs. Moreover, even though the accompanying dataset supports syntax-based learning, it lacks coverage of advanced design descriptions such as finite-state machines (FSMs), pipelined architectures, or memory-mapped modules, which can reduce the model's applicability. And, as with many recent LLM-based HDL tools, the framework primarily evaluates syntactic and token-level metrics.

Verigen [45] further contributes to the necessity of a proper dataset as it is a fine-tuned LLM explicitly optimized for Verilog code generation, achieving significant improvements in syntactic correctness and simulation-based pass rates over leading models. The authors fine-tune their model on a curated dataset sourced from GitHub repositories and educational materials, and evaluate across different Verilog generation tasks including module instantiation, conditional logic, and arithmetic design. Despite these advancements, however, *Verigen* has limitations that constrain its generalizability. The training dataset, while carefully filtered, is primarily composed of academic-level and textbook-style designs, which may not reflect the structural and functional complexity found in industry-grade RTL. Additionally, it often fails to comprehend natural language within prompts, resulting in hallucination within simple codes on its smaller models. *OriGen* [9] introduces a RTL code generation framework, emphasizing self-reflection capabilities and an innovative code-to-code augmentation methodology. By using knowledge distillation from commercial LLMs like Claude3-Haiku, *OriGen* refines open-source RTL code datasets to improve training quality. The framework employs a dual-model structure: Gen LoRA for initial code generation based on natural language instructions, and Fix LoRA for iterative error correction using compiler feedback. The drawback to this technique is that the augmentation strategies primarily focus on simple module patterns like adders and basic arithmetic units, limiting *OriGen*'s applicability to more complex or hierarchical designs.

BetterV [31] builds upon compiler-feedback approaches by integrating generative discriminators and reward models tied to PPA. It attempts to improve generation quality with hardware metrics. However, its evaluation focuses on synthetic test cases, with limited exploration of simulation, assertion-based validation, or deployment scenarios. *AutoVCoder* [12] introduces a two-stage fine-tuning pipeline combined with RAG to enhance both syntax and simulation-level correctness. It uses Verilog corpora along with code search to augment generation with applicable examples. This is effective in reducing syntax errors, but *AutoVCoder* does not verify generated outputs through computer-aided design (CAD) tools, making its use in high-assurance applications uncertain. *CodeV* [56] focuses on improving dataset quality by generating description-code pairs for training and summarization. This enhances prompt construction and provides

clearer supervision signals for LLMs during fine-tuning. However, the generated summaries are often generic, and the dataset lacks annotation for resource constraints or modular hierarchy. *CraftRTL* [23] integrates graphical inputs such as waveforms and FSM diagrams to support multimodal Verilog learning. The model leverages error feedback during training to improve code correctness. While promising in its use of visual-verbal alignment, the training data is limited, and there is little evidence of success on complex control logic or multi-module designs. Its utility in traditional design flows remains experimental.

2.3.3 Agentic Methods. Other approaches attempt to increase model performance through feedback mechanisms relying on integrating multiple LLM agents. *AutoChip* [46] provides a framework where an LLM such as GPT-4o produces the initial Verilog code based on a natural language instruction, for instance, “create a 4-bit adder.” The code is then assessed for syntax errors using a Verilog compiler, like Verilator [42], and for functional accuracy through testbenches. Additionally, a feedback loop allows the LLM to refine the code iteratively, continuing until the code is correct or an iteration limit is reached. While *AutoChip* demonstrates improved accuracy through automated refinement, it has limited applicability for large-scale hardware blocks. *PromptV* or *CooperativeV* [27] leveraged multi-agent prompting where multiple models handled different aspects of the code generation process. The goal is to create a collaborative yet competitive relationship between models to produce higher quality Verilog. With the increased overhead of many models, it requires considerations for scalability and deployment in resource-constrained environments. Also, high accuracy for complex hierarchical Verilog produced through integrating multiple LLM outputs is yet to be seen.

2.3.4 Benchmarks. *VerilogEval* [22] is a multi-stage benchmarking framework to assess the capabilities of LLMs in Verilog code generation in a structured manner. This framework evaluates models based on syntax, functional simulation, and formal verification, allowing for consistent comparisons and helping identify areas for improvement. The benchmark is still limited by the small scale of its dataset. The benchmark primarily targets self-contained Verilog modules such as adders, multiplexers, and basic control logic, which do not reflect the complexity or hierarchical depth of real-world hardware designs. As a result, models evaluated using *VerilogEval* may show strong results on these micro-benchmarks while struggling with larger, pipelined modules common in industrial RTL. Additionally, the dataset lacks diversity in coding style, parameterization, and integration with IP cores or testbenches, which are necessary for assessing generalization. These constraints limit the benchmark’s ability to capture practical challenges during synthesis, integration, or in the bitstream. Despite the limitations, *VerilogEval* is a great start for DB creation because it puts criteria in place for proper evaluation of LLM Verilog code generation. Our goal is to extend the current literature with an automated framework of curating, preprocessing, and storing a large high-quality Verilog dataset for effective GenAI fine-tuning. *RTLML* [25, 26] is a benchmark and evaluation framework that currently includes 50 hand-crafted designs that include hand-made RTL designs, their testbenches, and natural language descriptions. The evaluation framework facilitates syntax checks, functionality checks via simulations, and design quality checks (PPA) post-synthesis. These 50 designs, although high-quality, may not adequately test the full complexity and diversity of real world designs.

The field of LLM-driven Verilog generation has rapidly evolved from early prompt-engineering-based approaches toward more sophisticated frameworks that incorporate fine-tuning, compiler feedback, and benchmarking. Despite these advances, significant challenges remain, including small dataset sizes, limited support for complex hierarchical designs, and inconsistent verification of generated codes. The latest tools attempt to improve these drawbacks by exploring multimodal inputs, reward-based training, and graph-enhanced modeling. However, they still fall short in terms of real-world applicability, scalability, and full-system integration. This highlights the need for a large-scale

dataset curation framework that enables proper domain-specific fine-tuning or agentic approaches for more consistent efforts in RTL code generation.

3 VerilogDB

To allow scalable training and evaluation of LLMs in Verilog code, we developed *VerilogDB*, an end-to-end system that automates the collection, sanitization, annotation and structuring of Verilog RTL designs. This framework is designed to collect diverse sources of Verilog code, apply filtering and synthesis-aware validation, and output standardized metadata and instruction-friendly formats. The pipeline spans from raw repository scraping and PDF extraction to structured storage in a DB that can be queried. The following subsections detail each major component of *VerilogDB*, including our data collection strategy in Section 3.1, preprocessing filters in Section 3.2, and DB architecture in Section 3.3.

3.1 Data Collection

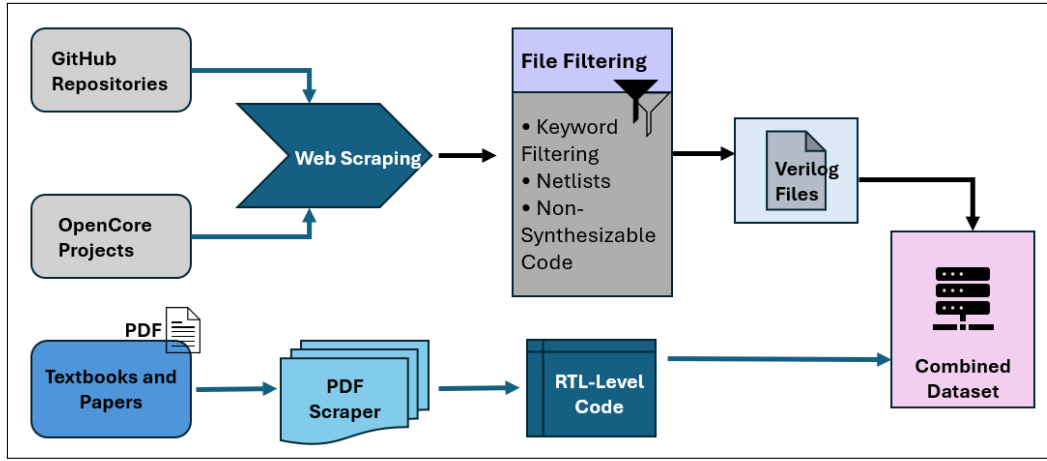


Fig. 2. Overview of the Verilog RTL data collection process from GitHub, OpenCores, and textbooks. Each source passes through a source-specific pipeline to extract syntactically valid and synthesizable RTL code.

A critical component in constructing a large, diverse, and high-quality dataset is curating the data samples adequately to meet the goal of consistent LLM generation of Verilog. To meet this requirement, we developed a data collection framework targeting publicly available Verilog repositories. Our sources included general-purpose hosting platforms (e.g., GitHub), hardware-specific archives (e.g., OpenCores), and educational repositories hosted on academic domains including and not limited to digital design textbooks. The overarching goal was to capture real-world design diversity while guaranteeing structural clarity before the preprocessing stage explained in Section 3.2.

3.1.1 GitHub Scraping. We queried GitHub through its REST API to identify repositories relevant to hardware design. Using keyword filters such as `verilog`, `rtl`, `fpga`, `hdl`, and `asic`, we retrieved a corpus of potential repositories from multiple sources and users. Each repository was then cloned, and only those containing Verilog source files (`.v`) were retained. To enforce relevance and quality, we implemented a branch-level and folder-level exclusion mechanism where branch names and directory paths containing generic or infrastructure-focused keywords were discarded to avoid including test harnesses, simulation environments, or system-level wrappers. Within each selected repository, Verilog

files were extracted along with accompanying documentation (e.g., README, license files, and comments). The data collected from GitHub amounted to more than ~30 GB of raw data that span a wide range of application domains, code styles, and project sizes, from single-module educational designs to hierarchical industrial IPs. After filtering, ~2 GB of Verilog data with around 72,600 Verilog modules are prepared and passed through the preprocessing pipeline.

3.1.2 OpenCores and Academic Sources. OpenCores [29] is one of the longest-standing archives of open-source HDL IPs and serves as a prominent source of synthesizable Verilog modules. We examined project pages across categories such as arithmetic cores, cryptographic primitives, and DSP blocks. Each project was downloaded and filtered through our standard exclusion criteria, targeting the elimination of:

- Files labeled as netlists or synthesis output (e.g., `_netlist.v`, `_synth.v`)
- Testbenches or simulation artifacts (e.g., files containing `$display`, `initial`, `waveform.vcd`, or `dumpfile`)
- Projects using unsupported language extensions (e.g., SystemVerilog in plain `.v` files)

Approximately 165 MB of raw Verilog was collected from OpenCores, which, after filtering, yielded 874 clean RTL modules suitable for synthesis and further annotation.

We further complemented our corpus by mining Verilog samples from academic course websites. These sources included assignment repositories, lab guides, and instructor repositories from top-ranked universities offering courses in digital logic or computer architecture. Although smaller in volume, these files are often well-commented and follow canonical design practices, making them ideal training examples. We extracted ~500KB of data samples from these sources.

3.1.3 Organization, Challenges, and Raw Statistics. All files were grouped by source and repository, preserving original folder hierarchies. This is significant for the later synthesis testing (discussed in Section 3.2.3). We maintained metadata for each project, including:

- Source of origin (e.g., GitHub, OpenCores, Academic)
- File type distributions (e.g., Verilog, testbench, config, README)
- Repository-level notes such as license, readme presence, and author information

One of the primary challenges encountered during this phase was the high variability in Verilog code quality and structure. Some repositories used informal or undocumented coding styles, lacked top-level modules, or contained extensive non-Verilog assets (e.g., C test drivers, synthesis scripts, IPXACT metadata). We used a permissive initial collection strategy to avoid early bias, deferring aggressive filtering to the preprocessing stage (Section 3.2).

In total, the aggregated raw dataset consists of:

- **GitHub:** ~30 GB pre-filtered across thousands of repositories and 2 GB filtered
- **OpenCores:** ~165 MB of raw Verilog IP cores
- **Academic sources:** ~500KB manually verified course-level RTL projects from top digital design curricula

This collection forms the foundation to our structured preprocessing pipeline, designed to provide high fidelity and LLM compatibility for instruction tuning and Verilog generation.

3.2 Data Preprocessing

The preprocessing pipeline addresses challenges in Verilog codebases, such as duplicate files, syntax errors, and synthesis issues, while extracting critical module metadata (e.g., module names, ports, comments, and token counts) and formatting

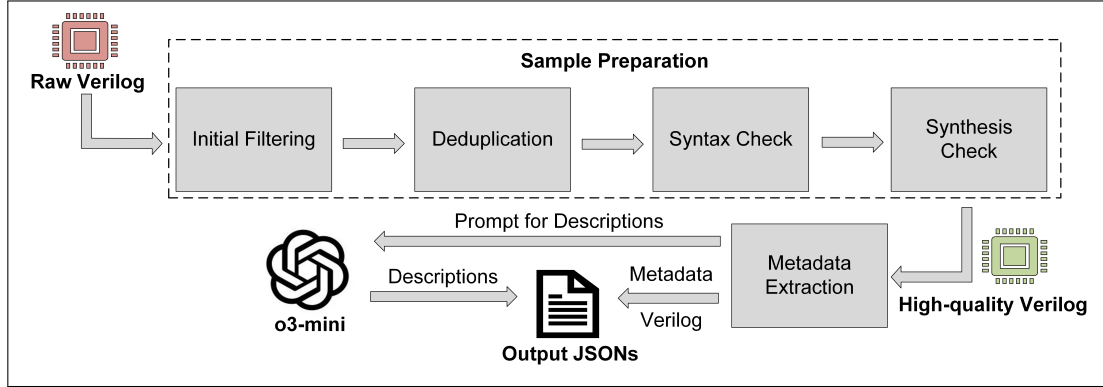


Fig. 3. Overview of the VerilogDB Framework Data Preprocessing Flow.

it into a JSON-based structure optimized for LLM consumption. The goal of this flow is to assure the quality of data samples for effective LLM fine tuning. Figure 3 illustrates the stages involved in the preprocessing pipeline of *VerilogDB*. The workflow consists of five primary steps: (1) initial file filtering, (2) duplicate removal, (3) syntax checking, (4) logic synthesis, and (5) metadata extraction. Each stage is designed to maintain data quality and compatibility with downstream LLM applications, such as code generation, hardware design analysis, or automated documentation.

3.2.1 Initial File Filtering. As mentioned in Section 3.1, a preliminary filtering removes files which do not fit the use case or to reduce processing overhead. If the files or project folders contain `*_netlist.v`, `*_gate.v`, `*_mapped.v`, or `*_synth.v`, they are removed from the initial sample as the dataset should only include valid Verilog modules useful for LLM fine tuning for RTL generation. Likewise, if the data contained `sim`, `simulate`, `waveform`, `_test`, they were also removed as we desire solely `.v` files which are synthesizable module files. The script excludes testbenches by filtering files containing "tb", "TB", or similar testbench nomenclature in their names and modules, as these typically include non-synthesizable constructs like delays or file I/O.

3.2.2 Deduplication. The second stage of the preprocessing pipeline focuses on eliminating duplicate Verilog files to reduce redundancy and improve processing efficiency in the later stages. Deduplication improves the quality of a dataset, reduces bias, and improves the performance of the model when fine-tuned or trained. The fine-tuning dataset must be generalizable and representative, avoiding concerns like overfitting and memorization. Duplicate Verilog files can arise in large HDL repositories due to version control artifacts, automated design tools, or copied modules. A custom Python deduplication script was built and deployed, utilizing MD5 hashing [33] for accurate detection of identical files based on their content, regardless of file names or locations. Algorithm 1 shows the *VerilogDB* deduplication process. MD5 was selected for its speed, ease of implementation, and efficiency. As seen in lines 2-13, the script first iterates through the designated project directory, computing a hash for each `.v` file and grouping files by their hash values. For each group of duplicates, the script retains the file with the shortest path (to prioritize original sources) and removes the others from subsequent processing as shown in line 15. The deduplication stage produces a unique set of Verilog files, minimizing computational overhead in subsequent stages and reducing overfitting in LLM training due to redundant data [20, 40].

3.2.3 Syntax Check. Following deduplication, the pipeline performs syntax checking to verify that Verilog files adhere to the HDL's grammatical rules. Syntax errors, such as missing semicolons, improper module declarations, and missing

```

Require: input_dir
Result: output_dir
// Initialize dictionary for storage of file paths by hash
1: hash_to_files ← empty dictionary;
// Step 1: Collect all .v files and compute their hashes
2: for file in recursive traversal of input_dir do
3:   if is_verilog_file(file) then
4:     file_path ← full path to file;
5:     file_hash ← ComputeMD5Hash(file_path);
6:     if file_hash exists in hash_to_files then
7:       | Append file_path to hash_to_files[file_hash]
8:     end
9:   else
10:    | hash_to_files[file_hash] ← new list containing file_path;
11:  end
12: end
13: end
// Step 2: Copy one file per unique hash to output directory
14: for file_hash, file_paths in hash_to_files do
15:   kept_file ← file_paths[0];
16:   rel_path ← relative path of kept_file with respect to input_dir;
17:   output_path ← join(output_dir, rel_path);
18:   // Create necessary directories
19:   Create directory structure for output_path if not exists
20:   // Copy file to output directory
21:   Copy kept_file to output_path with metadata preservation
22: end

```

Algorithm 1: Duplicate Removal Algorithm

module names in instantiations, can prevent successful compilation and synthesis, rendering files unsuitable for metadata extraction and for the dataset as a whole. A custom Python script utilizes the Icarus Verilog (iVerilog) compiler [48], a widely-used open-source tool, to verify each code’s syntax. It automated the syntax checking process, allowing for a quicker preprocessing flow than manual inspection of each data sample. Algorithm 2 shows the *VerilogDB* process for the syntax checking per each Verilog file. The script processes each .v file in the deduplicated directory, invoking iVerilog in subprocesses, seen in lines 2-3, to check syntax for each file without generating output binaries. The goal of this stage is syntax check and not elaboration, which will be verified in the later synthesizability check stage. Responses from iVerilog help indicate the error/warning types. Moreover, the script properly isolates syntax errors from elaboration errors, creating a list of the files which pass syntax and ignores the elaboration errors as we do not check synthesis yet to increase processing efficiency. This can be seen in lines 17-38. These files that pass the syntax check are copied to an output directory, while errors are logged with details of the failure (e.g., line numbers and error messages).

3.2.4 Synthesizability Check. After cleaning the data samples from netlists, testbenches, duplicates, and syntactically-incorrect code, the third stage verifies the synthesizability of the Verilog files. Synthesis checking is critical for LLM applications targeting hardware design, as non-synthesizable codes present in the dataset will skew LLMs’ fine-tuning or training away from generating code that can be used for a physical device. We developed a script that employs Yosys [49], an open-source synthesis tool, to perform synthesis checks on each subproject. Algorithm 3 shows the

```

Require: file_path
Result: success
// Verify tool and file exist
1: Leave algorithm if tools and file does not exist.
// Run Icarus Verilog with syntax check options
2: cmd ← [IVERILOG_PATH, "-o", "nul", "-Wall", file_path];
3: result ← execute cmd, capture stdout and stderr as text;
4: stderr_output ← strip result.stderr;
// Check for clean pass
5: if result.returncode = 0 then
6:   if stderr_output is not empty then
7:     Log "Warnings detected: stderr_output"
8:     return True, "Syntax check passed (with warnings: stderr_output)"
9:   end
10:  else
11:    return True, "Syntax check passed"
12:  end
13: end
// Analyze errors
14: error_lines ← split stderr_output into lines;
15: has_syntax_error ← False;
16: elaboration_errors ← empty list;
// Define error patterns
17: syntax_error_pattern ← regex matching syntax error outputs from iverilog;
18: elaboration_error_pattern ← regex matching elaboration error outputs from iverilog;
19: for line in error_lines do
20:   if syntax_error_pattern matches line then
21:     has_syntax_error ← True;
22:   end
23:   else if elaboration_error_pattern matches line or "warning: macro" in line then
24:     Append line to elaboration_errors
25:   end
26: end
// Determine outcome
27: if has_syntax_error and elaboration_errors is empty then
28:   error_message ← stderr_output or "Unknown syntax error";
29:   return False, "Syntax error: error_message"
30: end
31: else if elaboration_errors is not empty then
32:   error_message ← join elaboration_errors with newlines;
33:   return True, "Syntax OK, elaboration issues: error_message"
34: end
35: else
36:   error_message ← stderr_output or "Unknown error";
37:   return False, "Check failed: error_message"
38: end

```

Algorithm 2: Syntax Check Algorithm per .v File

VerilogDB process for checking synthesizability per project. The script operates at the repository/project-level with the goal of testing multiple files at once for synthesizability, as Yosys facilitates batch processing projects for logic synthesis which is the standard for these tools.

Firstly, a top-level module is identified from the subproject, finding the hierarchy (lines 2-3). The script does this by first checking the files with "top" in the module or file name and then iteratively checking for which file in a project will synthesize as the top-level. This iteration is necessary as many times top modules do not have "top" in the file name. As seen in lines 4-12, each syntax-checked file within the scope of a subproject is batch processed using Yosys's synth command, which attempts to map the Verilog code to a generic gate-level netlist. Files in projects that are successfully synthesized are passed to the metadata extraction stage and failures are logged with diagnostic information. We also note that there were cases where projects had files with the same module names which the script handles by checking synthesizability of all possible scenarios of modules with that same name. For example, a project may have two modules named *counter* but one counts to 256 than the other 512. The script checks synthesizability for each case where either counter is instantiated in the design.

```

Require: files_in_proj
Result: failure, output
// Setup and tool/file check
1: Leave algorithm if Yosys and file does not exist.
// Identify unique modules
2: unique_files ← GetUniqueModules(files_in_proj);
3: top_module ← DetermineTopModule(unique_files);
// Build Yosys script
4: script ← empty string;
5: for file in unique_files do
6: | Append "read_verilog \"file\"" to script
7: end
8: Append "hierarchy -check -top top_module" to script
9: Append "synth -top top_module" to script
// Run Yosys
10: cmd ← [YOSYS_PATH, "-q", "-p", script];
11: result ← execute cmd, capture stdout and stderr as text;
12: output ← strip(result.stdout) + newline() + strip(result.stderr);
// Analyze Errors
13: errors ← matches of regex pattern for non-tolerable Yosys error/warning outputs;
14: if errors is empty then
15: | return False, output // No Errors, Synthesis Passed
16: end
17: else
18: | return True, output // Errors Present, Synthesis Failed
19: end

```

Algorithm 3: Synthesis Check Algorithm per Project

3.2.5 Metadata Extraction. The metadata extraction stage for sample preparation, implemented in a custom Python script, processes the synthesizable Verilog files, capturing specific metadata from each module including the module name, port names and widths, comments, estimated LLM token count, and a high-level natural language description of

```

{
  "module_name": "dec",
  "ports": [
    {
      "name": "I",
      "direction": "input",
      "bit_width": 2
    },
    {
      "name": "v",
      "direction": "input",
      "bit_width": 1
    },
    {
      "name": "y",
      "direction": "output",
      "bit_width": 4
    }
  ],
  "comments": [],
  "verilog_code": "module dec (\n input [1:0] I,\n input v,\n output reg [3:0] y\n);\n\n  always@(I)\n    begin\n      case({I,v})\n        3'b001: y = 4'b0001;\n        3'b011: y = 4'b0010;\n        3'b101: y = 4'b0100;\n        3'b111: y = 4'b1000;\n        default: y=4'b0000;\n      endcase\n    end\n  endmodule",
  "token_count": 35,
  "description": "This module decodes a 2-bit input and control signal into one of four 4-bit outputs."
}

```

Fig. 4. Data sample JSON for decoder containing metadata and generated description.

the code. The primary challenge of this stage is parsing diverse Verilog syntax to capture essential module information, including module names, port declarations, comments, parameters, and estimated token counts. The script employs regular expressions (regex) to robustly handle Verilog constructs, ensuring compatibility with different coding styles and edge cases.

- (1) *Regex-based Parsing* - Firstly, modules are identified in the .v file. A regex pattern `MODULE_RE` matches module declarations while ignoring commented-out modules (e.g., `//module ...`) using negative lookbehinds (`((?<![/*])`). This ensures only active modules are processed, addressing issues where commented code may be mistakenly parsed. Also, in the module declaration, whitespace is taken into account as the definitions may differ in the use of whitespace. We did not normalize the whitespace in preprocessing because we wish for the LLM to understand the variety of spacing styles and indentation practices. It also identifies cases where a Verilog file has multiple modules. Next, a regex pattern `PORT_BLOCK_RE` captures input, output, and inout ports, supporting both ANSI and Non-ANSI styles and the various port definition formats (e.g., comma-separated port definitions, bit-widths with or without spaces, etc) and keywords (e.g., `wire`, `reg`, `signed`). Comments are also extracted with a regex pattern `COMMENT_RE` identifying single-line (`// ...`) and multi-line (`/* ... */`) comments. In the case where the module definition has parameter declarations (e.g., `parameter WIDTH = 8;`) for bit-width calculations, these are identified with `PARAMETER_RE` regex pattern.
- (2) *Token Estimation and Description Generation* - Aside from a conventional Verilog parser/tokenizer which are syntax-aware and may not be accurate to how LLMs interpret tokens, we developed a custom tokenizer to

estimate the number of tokens as LLMs would (e.g. handling identifier names, whitespace, and punctuation). The arduous process of annotating module descriptions was also automated to greatly scale the DB using OpenAI's o3-mini to generate for every module, a corresponding singular sentence description with a maximum of 40 words. A short description was utilized as to provide the LLM with a concise and efficient description. We prompted o3-mini with the following:

- **Description Prompt to o3-mini:** Describe what the following Verilog code does in 40 words or less, ending with a period: `\n\n"verilog_code\n\n"` Focus on the module's core function.

(3) *Output Structure* - The extracted metadata is formatted as JSON files with one file per module (e.g., AES_Core.json). Figure 4 shows an example outputted JSON for a D flip flop Verilog file d_ff.v. Each JSON includes:

- *module name:* The extracted name of the module.
- *ports:* A list of dictionaries with name, direction, and bit_width (e.g., [{"name": "din", "direction": "input", "bit_width": 128, ...}]).
- *comments:* A list of cleaned comments.
- *Verilog code:* The full module code, preserving original formatting and comments.
- *token count:* An estimate of code complexity based on whitespace and separator splitting. Useful for LLM fine-tuning.
- *description:* The module's generated description from o3-mini.

3.3 Database Architecture

With the sheer volume of needed Verilog data, a DB hosted on dedicated hardware with sufficient storage is a key part of our framework.

3.3.1 *Database Selection.* PostgreSQL [14] is our chosen platform for hosting the *VerilogDB* dataset due to the following reasons:

- (1) *Structured and Semi-Structured Data Support* - It handles exceptionally structured data (e.g., where module metadata is stored in tables) and semi-structured data (e.g., where module metadata is stored directly as JSONs). The *VerilogDB* preprocessing flow generates JSON metadata, which PostgreSQL can efficiently store using its native JSONB data type.
- (2) *Verilog Module Querying Capabilities* - It supports complex SQL queries, including joins, aggregations, and window functions, which are essential for analyzing *VerilogDB*'s dataset (e.g., grouping by module complexity) as its full-text search and JSONB querying is an efficient retrieval of prompt-response pair instructions for LLM fine-tuning.
- (3) *Scalability and Performance* - As it can scale vertically, PostgreSQL benefits *VerilogDB* with its 20,392 module entries as the system handles large datasets efficiently with features like parallel query execution, indexing (e.g., B-tree, GIN for JSONB), and partitioning.
- (4) *Data Integrity and Reliability* - Through constraints (e.g., primary keys, foreign keys, unique constraints), PostgreSQL enforces strong data integrity, guaranteeing that module metadata is consistent and reliable. Its ACID (Atomicity, Consistency, Isolation, Durability) compliance assists data management to preserve dataset quality during LLM fine-tuning.

```

CREATE TABLE verilog_modules (
    id SERIAL PRIMARY KEY,
    module_name TEXT NOT NULL,
    verilog_code TEXT NOT NULL,
    description TEXT NOT NULL,
    comments TEXT,
    token_count INTEGER,
    extracted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Table to store ports for each module
CREATE TABLE module_ports (
    id SERIAL PRIMARY KEY,
    module_id INTEGER REFERENCES verilog_modules(id) ON DELETE CASCADE,
    port_name TEXT NOT NULL,
    port_type TEXT CHECK (port_type IN ('input', 'output', 'inout')) NOT NULL,
    port_width INTEGER,
    CONSTRAINT unique_port UNIQUE (module_id, port_name)
);

```

Fig. 5. SQL schema for PostgreSQL structured DB Implementation

3.3.2 DB Structure and Sample Insertion. As mentioned in Section 3.3.1, the PostgreSQL implementation can support both structured and semi-structured data storage. Figure 5 shows the SQL schema for the structured data storage implementation using PostgreSQL. Primary keys for the module IDs and port IDs provide unique identifiers that are B-tree indexed [6], allowing for efficient lookups and joins that are critical for the retrieval of specific modules or the linkage of ports to modules. Data integrity is enforced where certain metadata (e.g., module names, descriptions, and port names) is required to be present. In the module_ports table, each port in a module must be unique, further enforcing the Verilog structural rules. Comments from the Verilog codes are left as optional in the DB, as designers may or may not annotate their modules with comments.

Automated scripts were developed to insert the JSON contents into the PostgreSQL DB tables in a streamlined fashion. Due to the selection of PostgreSQL, a variety of significant validation logic is inherent for proper metadata construction before DB insertion. We use these innate integrity benefits of the PostgreSQL DB to verify the metadata extraction and deny any insertion of improper entries. As ports are enforced to not be null before the module’s insertion into the DB, any module where its ports or module definition were not captured by the regex patterns is denied entry into the DB. Any attempt to insert identical data to an entry in the DB is denied due to an added unique constraint on the verilog_code column. Certain exceptions are made for modules that are meant to have no ports. For example, modules that are standard wrappers for fills or decoupling capacitors may contain no ports.

3.4 Sample Preparation for Instruction-Tuning

JSON data for fine-tuning is designed to support LLM tasks such as code completion, design optimization, and automated documentation. The data can also aid in our purpose of RTL code generation. As a key final step for fine-tuning, we formatted the extracted metadata for LLM consumption via a custom Python script, which processes DB samples, providing compatibility with LLM training pipelines by formatting them into proper instructions [4, 30]. LLM instruction-tuning requires the data to be formatted in prompt-response pairs. For example, to instruction-tune an LLM that can effectively translate English to Spanish, the prompt would be to translate a phrase with the associated response as the translation. For the use case of Verilog code generation, the prompt would ask for a design following a set of

specifications with the response being the code itself. The construction of the prompt component lies in the metadata. For each module prompt, we first place a consistent system prompt for the LLM [38, 39], then prompt the LLM to generate a Verilog module named `module_name`, including the ports and description of the module. Below is an example prompt and response for an adder circuit:

Example Prompt $\mathbb{P}_1$: You are a highly experienced RTL code designer skilled at designing concise, syntactically correct, and synthesizable Verilog code that functions.

Generate Verilog code for a module named ADD with the following ports and description:

input [15:0] in1, input [15:0] in2, output [15:0] out

This module performs combinational addition, computing the sum of two 16-bit inputs to produce a 16-bit output.

Example Response $\mathbb{R}_1$:

```
module ADD (out, in1, in2);
input [15:0] in1, in2;
output reg [15:0] out;
always @(*)
out = in1 + in2;
endmodule
```

With the stored estimated token count value for each data element, data samples for specific context window sizes can be pulled. For example, for Mistral-7B [18] and Code Llama-7B [34], the context windows are 8,192 and 4,096 tokens respectively. This implies that Verilog code samples should ideally be below those token counts, as an LLM interpreting a truncated input code leads to a loss of information/context and thus degrades performance in many scenarios [41, 52]. *VerilogDB*'s distribution of code based on token count is discussed and shown in Section 4.

4 Analysis

We passed our ~30 GB raw data through our preprocessing pipeline (see Figure 3), where the sample cleaning metrics per stage can be seen in Table 1. We note an effective processing flow yielding 751 MB of total output data size and 20,392 Verilog modules prepared for further LLM fine-tuning or other downstream tasks. In addition to an analysis on the preprocessing efficacy, we also assess *VerilogDB*'s data diversity along four key aspects: 1) Functional categories, 2) Module Complexity, and 3) Annotation and Comments. We also provide a discussion on the benefits and challenges of *VerilogDB*.

4.1 Preprocessing Evaluation

After 2 GB, about 72,600 modules, passed initial filtering, this subset was passed through our remaining preprocessing pipeline. Approximately 12.47% of the 72,600 modules were found to be exact duplicates and were removed, reducing the size of the dataset without losing unique design information. In our experiments, 63.07% of deduplicated modules passed syntax checking, with failures primarily due to incomplete modules or non-standard Verilog constructs, which were excluded from further processing to maintain data quality. In our dataset, after removing testbenches and files with non-synthesizable constructs, approximately 69.49% of syntax-checked modules passed synthesis. Common failures were attributed to unsupported constructs or missing dependencies, which were flagged for manual inspection. Metadata extraction was done successfully for over 99.01% synthesized modules, with failures mainly due to improper port

Table 1. Verilog Preprocessing Pipeline Stages and Filtering Statistics

Stage	Description	Input (MB)	Output (MB)	% Retained	Common Rejection Reasons
Deduplication	Removes textually identical modules	2000	1750	87.53%	Identical files
Syntax Check	Drops modules with syntactic errors during parsing	1750	1104	63.07%	Lexical issues, malformed declarations, incorrect use of blocking assignments
Synthesis Check	Passes only synthesizable modules	1104	767	69.49%	Unsupported logic, invalid constructs, use of delays
DB Validation	Ensures compatibility with internal database schema	767	751	99.01%	Schema mismatch, unresolved symbols, incorrectly parsed metadata

declarations and unidentified modules. These were caught and prevented for insertion by the DB validation logic. In our experiments, the preprocessing pipeline outputted a dataset of 751 MB of high-quality Verilog data, producing 20,392 unique, synthesizable modules with complete metadata, demonstrating diversity and scalability.

4.2 Dataset Distribution and Diversity

After deduplication and filtering, we examined the distribution and composition of the resulting Verilog dataset along functional, structural, and annotation-based axes. Our goal was to guarantee the retained modules reflect a wide variety of RTL design styles and application domains and span an array of sizes/complexity.

4.2.1 Functional Categories. We observed a broad representation of modules across key RTL design classes. These include arithmetic blocks (e.g., adders, multipliers, ALUs), cryptographic components (e.g., AES rounds, modular multipliers), DSP modules (e.g., FIR filters, MAC units), control logic (e.g., counters, FSMs, encoders), and memory interfaces (e.g., register files, FIFOs, RAM controllers). This functional diversity is important for exposing LLMs to different types of datapaths, control patterns, and signal-level interactions typical in modern SoC designs. In order to adequately analyze the functional makeup of the dataset, we passed all *VerilogDB*'s modules to o3-mini to classify the modules into 13 different classes based on module type/functionality. Figure 6 shows the distribution by functional class for the modules in *VerilogDB*. The class numbers, names, and example modules are listed in Table 3 in the Appendix. Most modules in *VerilogDB* fall under Class 1, Basic Digital Building Blocks, that total 46.17% modules. Classes 2 and 3, Combinational and Sequential Logic Designs, make up > 40% of the *VerilogDB* dataset. The remaining ~12% of the dataset is well distributed across the rest of the classes spanning from custom accelerators to security and crypto blocks. Provided the large number of data, 761 MB of Verilog data, oversampling minority classes or loss reweighting [10] can be implemented to prepare a more balanced sample for LLM fine-tuning.

4.2.2 Module Complexity. The dataset spans a wide spectrum of design scales. Some modules are compact, representing atomic components or student-written designs for educational use. Others are larger and structurally complex, composed of multiple submodules with layered hierarchy and interconnect logic. This range provides LLMs with examples for learning both isolated RTL constructs and compositional hardware behaviors. We utilize three key metrics to approximate

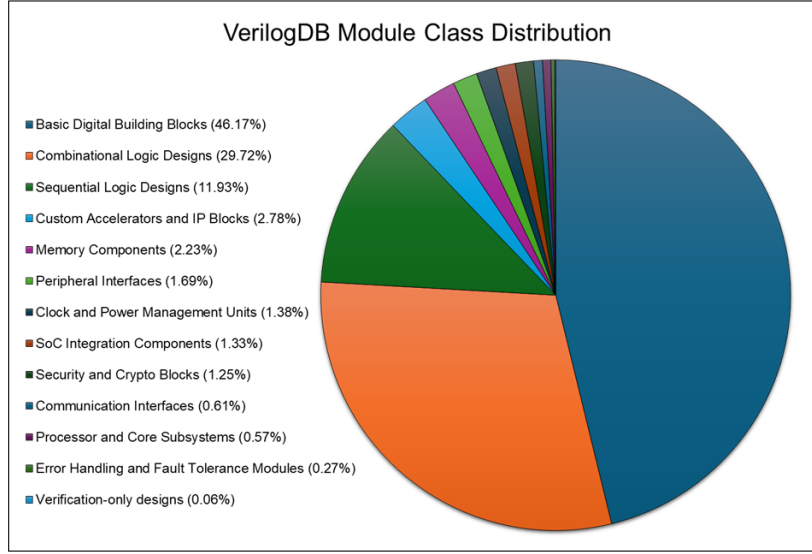


Fig. 6. Class Distribution of *VerilogDB* modules.

a module’s complexity: (1) estimated LLM token count to represent the module’s size for downstream processing, (2) Verilog code line count (not including blank lines) to highlight how large the file was, and (3) the number of ports in the module. In general, more complex and larger modules would take up more lines of Verilog code and take more LLM tokens while typically having more I/O ports. We also note further analysis can be performed to estimate the module complexity like counting the number of *always* blocks or *case* statements per code or how large *for generate* blocks will unravel, as for small line counts they can still relate to large complex designs. As seen in Figures 7 and 8, most modules in *VerilogDB* are below 100 lines of Verilog code and are below 200 LLM tokens (15,475 modules). Also, most designs (17,715 modules) have ports from 0-10 with a port count mean μ_{pc} of 7.99 and median M_{pc} (better adjusted for non-symmetrical data) of 4. The LLM token count has a mean μ_{tc} of 2126.34 and median M_{tc} of 55, while the line count has a mean μ_{lc} of 700.16 and a median M_{lc} of 28. The higher mean values compared to medians indicate that the dataset is right/positively skewed and not symmetrical, indicating the presence of fewer larger and more complex modules making the dataset more tailored for smaller to moderate designs, essential for the foundation of hierarchical designs. Sampling for LLM fine-tuning must take this into account to select a well-balanced distribution of the modules from the dataset to account for the skew. In future work, we seek to expand the dataset’s range through data augmentation to address and mitigate biases.

4.2.3 Annotation and Comments. A subset of the dataset, particularly those sourced from academic and textbook-derived content, contains meaningful inline comments and block-level summaries. When available, these annotations are preserved and extracted as auxiliary metadata for instruction generation and semantic indexing. While not uniformly present, their inclusion supports higher-quality prompt creation for instruction-tuned LLMs. In *VerilogDB*, 53.9% of the modules (11,009 modules) have comments. A larger subset of modules with comments would further enhance the quality understanding of the code. We seek to develop automated code commenting in future work to enhance the dataset and improve this component, as fine-tuning an LLM with Verilog code containing comments provides more context and can improve model ability [15, 43]. A critical metric to evaluate the quality of a codebase’s commenting is code comment density [5], defined as the percentage of comment lines of code in a codebase. We define comment density

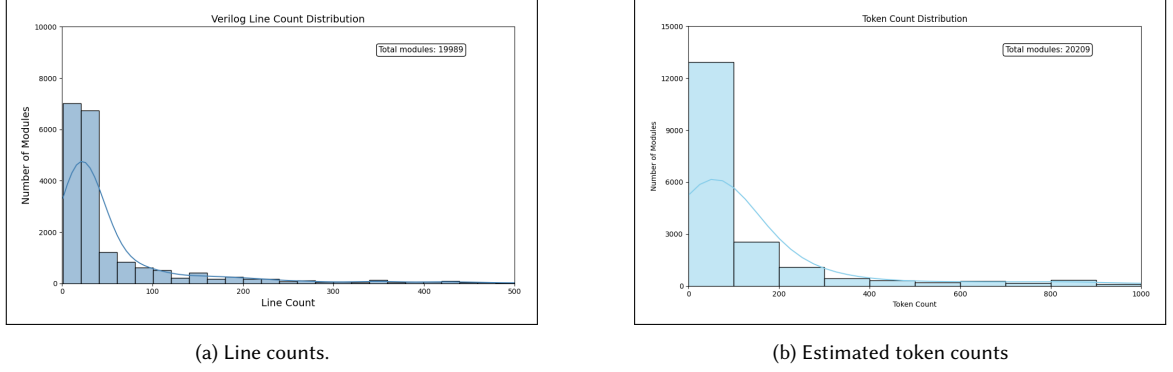


Fig. 7. Distributions of *VerilogDB* modules by (a) functional classes, (b) line count, and (c) estimated token count.

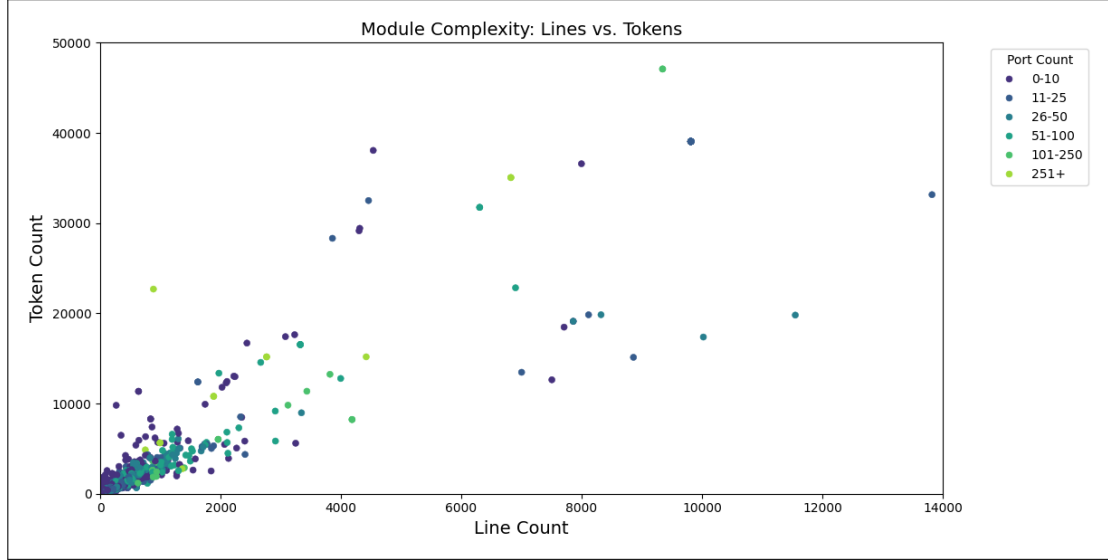


Fig. 8. Scatterplot showing of module complexity relating estimated token and line counts of the Verilog modules

as the number of comment characters over the total characters of the Verilog code. Figure 9 shows the distribution of modules based on ranges of comment densities. Of the 11,009 commented modules, 60.08% of the modules have a code comment density D_c where $0\% < D_c \leq 25\%$. Modules where $25\% < D_c \leq 50\%$ account for 11.92% of the 11,009 commented modules. Lastly, 3,079 modules (27.97% of commented codes and 15.1% of total *VerilogDB*) have $D_c > 50\%$. This reveals almost a third of the commented codes have more than half of the text coming from comments which are rich in context for LLM fine-tuning. Of the codes with comments, the mean μ_{D_c} is 1093.71% shows the wide range of comment densities. The median comment density M_{D_c} is 15.52%, providing a more accurate view of the average D_c that accounts for bias.

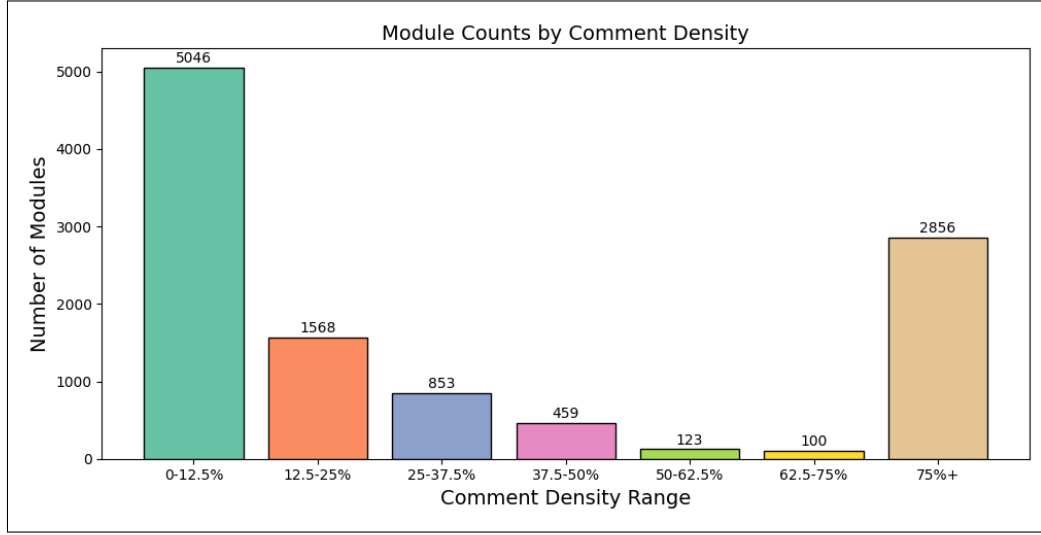


Fig. 9. Comment density module counts for 0-12.5%, 12.5%-25%, 25%-37.5%, 37.5%-50%, 50%-62.5%, 62.5%-75%, and >75% buckets

4.3 Discussion

The construction and analysis of our *VerilogDB* framework revealed a number of important benefits and challenges with VerilogDB’s dataset curation, preprocessing workflows, and implications for downstream model development. These findings inform both our current pipeline design and future directions for scaling Verilog-focused LLM research. While our approach in the current framework focuses on a dataset for LLM fine-tuning, we also note that *VerilogDB* can be leveraged for RAG-based approaches [13, 16, 19, 51] rather than fine-tuning; however, an additional vectorization stage would be necessary to produce the embeddings for queries. Table 2 provides an overview of key insights from our approach and their implications on LLM fine-tuning. We discuss both the benefits of *VerilogDB* compared to other works and the encountered challenges:

4.3.1 Benefits. Our dataset’s potential to augment automated hardware design is underscored through the following key points:

- **A Rich and Diverse Dataset Enhances HDL Research.** As shown in Figures 6 and 8, the dataset captures a variety of module types, interface complexities, and design styles. Arising naturally from multi-source scraping, this variation increases LLM efficiency and helps avoid overfitting to any single design idiom. Other works use datasets with Verilog modules which have been intentionally restricted to stay within line count ranges, limiting the capabilities of trained LLMs and their generalization to varying module complexities and functionalities.
- **Rigorous Preprocessing Augments Quality.** Exact and near-duplicate modules were commonly found across GitHub repositories, especially in forked or template-based designs. Removing them reduced redundancy, improved design diversity, and minimized the risk of LLM overfitting to high-frequency samples. While syntax checking is a common filtering step, it proved insufficient on its own as many syntactically valid files failed synthesis due to unresolved submodules, macro dependencies, or simulation-only constructs. Our combination of syntax and synthesis checks, paired with metadata extraction, provides a more reliable foundation for model training. Other works include preprocessing in a more manual fashion with little automation or rely on LLMs

to preprocess the data (syntax and synthesis checks) which are prone to hallucination and are not as robust as industry aligning compilation and logic synthesis tools.

- **Metadata and comment presence enable semantic alignment.** While the quantity and consistency of comments vary, their presence in over half of the dataset makes them valuable for prompt-response generation and instruction-tuning. This gives an opportunity for future work in automated comment enrichment. Other Verilog datasets for LLMs, however, widely do not consider comments as significant aspects to have span most of the dataset for context enrichment. In contrast, our approach stores all comment information as metadata.
- **LLMs must learn from real-world RTL noise.** Exposure to inconsistent formatting, incomplete modules, and stylistic variation in early preprocessing steps reinforces the need to train models not just on idealized textbook code, but also on the kinds of imperfect RTL encountered in practice. A small number of modules with excessive ports, unusual formatting, or partial declarations can serve as test cases to improve the model’s robustness. Other works remove whitespaces of modules or alter the data to match consistent stylings/formats as a preprocessing step to lead to higher accuracy for modules with that same formatting; however, their capabilities to generate or understand Verilog with a wide diversity of stylings, expected in real-world applications, is yet to be seen.

Table 2. Summary of Verilog dataset sources and their contribution to LLM training.

Insight Area	What We Observed	Concrete Examples / Evidence	Implications for LLM-Based RTL Learning
Preprocessing Effectiveness	Major size reduction from raw to clean data.	~30 GB raw → 751 MB cleaned; 20,392 usable modules.	Filters out noise, reducing overfitting and improving convergence.
Deduplication & Validation	High duplicate rate; multi-step filtering needed.	12.47% duplicates removed; syntax pass = 63.07%, synthesis pass = 69.49%.	Validations ensure training stability and correctness.
Module Complexity & Size	Range from simple to deeply nested modules.	Most <200 tokens and 100 lines; some exceed 10k tokens.	Models learn from basic logic to complex SoC components.
I/O Port Distribution	Wide variation in port counts.	Most modules: 0–10 ports; some >500 (Figure 4b).	Exposes models to diverse interface widths.
Comment Availability	Many include inline documentation.	53.9% of modules (11,009) have comments; 28% are over 50% comments.	Boosts instruction tuning with rich prompt-target pairs.
Metadata Extraction Success	Structured formatting after synthesis pass.	99.01% converted to JSON with ports, tokens, source.	Enables prompt templates and supervision pipelines.
Functional RTL Diversity	Broad spectrum of design types.	Modules span 13 RTL classes (Figure 6).	Supports generalization across hardware domains.
Design Source Heterogeneity	Sourced from varied ecosystems.	GitHub (varied), OpenCores (reusable IP), Academia (clean, instructional).	Enhances LLM robustness to style, naming, and structure.

4.3.2 *Challenges.* In developing *VerilogDB*, we encountered a variety of challenges:

- **Data Collection and Heterogeneity:** Collecting 20,392 samples from diverse repositories posed significant challenges due to the heterogeneity in coding styles, lack of adequate documentation, and inconsistencies in module formatting. Variations in Verilog syntax and inconsistent naming conventions complicated metadata extraction where our implemented regex patterns had to cover all possible module definition and port definition constructs. Validation was required to resolve ambiguities, increasing preprocessing time.

- **Preprocessing Scalability:** The preprocessing pipeline, while effective, was resource-intensive for the large sample set including some very large modules. Synthesis validation, which required Yosys, was particularly time-consuming and took days to complete. Scaling to larger datasets (e.g. 100,000 samples) would necessitate hardware intervention to improve performance, such as better memory optimization, distributed computation for parallelization, and high-core count CPUs. Furthermore, description assignment relied on LLM-generated summaries from o3-mini, which also took days to complete and occasionally produced generic or inaccurate descriptions, requiring review for critical modules during the DB validation stage.
- **Future Data Relevance:** The rapid evolution of hardware design practices and the adoption of newer standards pose a challenge to *VerilogDB*'s long-term relevance. While the 20,392 samples capture a snapshot of open-source Verilog modules, they may be based on older coding standards, potentially limiting their applicability to modern design flows. Continuously updating the dataset to include contemporary designs requires ongoing data collection and preprocessing, which is resource-intensive. This challenge underscores the need for a sustainable maintenance strategy to keep the DB aligned with industry trends, hence why our VerilogDB emphasizes DB creation and maintenance as a core pillar of the work, which other works do not discuss.

4.3.3 Using VerilogDB in DeepC. *VerilogDB* also provides an effective DB for RAG-based RTL generation approaches. As mentioned above, current RTL generators use RAG or agentic approaches; however, none incorporate both to optimize performance. Our current work, DeepC, leverages *VerilogDB* as the input for vector DB generation, enabling the retrieval system to have a large high-quality sample set to improve the model context. DeepC includes various agents for code correction, simulation/testbench generation and error checking, and code generation utilizing the vectorized *VerilogDB* through RAG. With our method, agents can leverage state-of-the-art LLMs as model architectures improve over time. In addition, RAG allows for more up-to-date Verilog codes to be added to the vector DB enhancing relevancy and facilitating industry partners to add proprietary IP. DeepC will be released soon.

5 Conclusion and Future Work

As LLMs have seen rapid growth in usage across most industrial sectors, particularly in technology, they can improve the acceleration of the hardware design flow. In our work, we showcase *VerilogDB*, a framework for developing a custom Verilog dataset for LLM fine-tuning for RTL code generation. We evaluated our three-part approach that involved data curation, preprocessing, and custom DB architecture and storage with a raw input dataset of ~ 30 GB to produce more than 751 MB of high-quality Verilog data. This large dataset With *VerilogDB*, we intend to expand the dataset and develop fine-tuning and RAG agentic approaches for HDL generation. We also seek to label the data by functionality, further enhancing module context for LLMs. Lastly, another critical goal for future work includes LLM secure-RTL generation for hardware IPs to meet evolving security specifications in a semiconductor space rife with hardware threats.

Appendix

Table 3. Class names and Definitions for VerilogDB Modules

#	Class Name	Example Components
1	Basic Digital Building Blocks	Multiplexers, Demultiplexers, Decoders, Encoders, Half Adders, Full Adders, Subtractors, Comparators, Parity Generators, Priority Encoders, Registers (D, T, JK, SR), Shift Registers (SISO, SIPO, PISO, PIPO), Binary Counters, Ring Counters, Johnson Counters
2	Combinational Logic Designs	Combinational ALUs (add, subtract, AND, OR, XOR, NOT, shift), Logic Gates (AND, OR, NOT, NAND, NOR, XOR, XNOR), Bit-slice architectures, Barrel Shifters, Carry Lookahead Adders, Booth Multipliers, Wallace Tree Multipliers, Binary to Gray and Gray to Binary Converters
3	Sequential Logic Designs	Finite State Machines (Moore, Mealy), Pipelined Data Paths (IF, ID, EX, MEM, WB stages), Clock Dividers, Timers, Watchdogs, Register Files with Read/Write Ports, Sequential ALUs with pipelined operation, Sequence Detectors, Pulse Stretchers, Edge Detectors
4	Memory Components	SRAM Controllers, DRAM Controllers (LPDDR4, DDR3/4), ROMs (fixed, OTP), EEPROM controllers, FIFOs (synchronous/asynchronous), Dual-Port RAMs, CAMs (Content Addressable Memories), Cache Controllers (write-through, write-back), Memory Protection Units (MPUs), Page Table Walkers
5	Communication Interfaces	UART, SPI (Full-duplex, Quad SPI), I2C (Standard, Fast, High-Speed), CAN, USB 2.0/3.0, PCIe PHY/MAC, AXI Interconnects, AHB Masters/Slaves, APB bridges, DMA Engines (single and burst transfer), Ethernet MAC/PHY (10/100/1000 Mbps), JTAG TAP Controllers
6	Security and Crypto Blocks	AES (128/256-bit), RSA Encryption/Decryption, SHA-1/SHA-2 Hash Engines, HMAC, True Random Number Generators (TRNGs), Pseudo-Random Number Generators (PRNGs), Secure Boot Engines, Key Management Units, Access Control Checkers, Secure Debug Locks
7	SoC Integration Components	Bus Arbiters (round-robin, priority), Crossbar Switches, Address Decoders, Interrupt Controllers (e.g., PLIC), Reset Controllers, Clock Domain Crossing (CDC) Synchronizers, Power Management Units, Voltage Regulators, System Control Registers, Boot ROM Mappers
8	Processor and Core Subsystems	Custom RISC-V / ARM cores, Microcode Engines, Instruction Fetch/Decode Units, Register Renaming Units, Reorder Buffers, Branch Target Buffers, Return Address Stacks, TLBs (Translation Lookaside Buffers), L1/L2/L3 Cache Hierarchies, MMUs (Memory Management Units)

#	Class Name	Example Components
9	Peripheral Interfaces	GPIOs (bidirectional, open-drain), PWM Generators, ADC Interface Controllers, DAC Control Logic, Touchscreen Interfaces (resistive/capacitive), Camera Sensor Interfaces (MIPI CSI), Display Interfaces (HDMI, VGA, LVDS), SD/MMC Card Interfaces, Audio Codecs (I2S, AC'97)
10	Verification-Only Designs	Basic Testbenches, Coverage Monitors (functional/code), Scoreboards for result checking, Bus Functional Models (BFMs), Interface Monitors, Checkers for protocol compliance, Reference Models
11	Custom Accelerators and IP Blocks	ML Accelerators (Matrix Multipliers, Softmax, Activation Units), DSP Units (FIR, IIR, FFT, DCT), Video Codecs (H.264, H.265), Neural Network Inference Cores, JPEG Compression/Decompression, Hardware Sorting Units, CRC Generators, Hardware Schedulers
12	Clock and Power Management Units	Clock Gating Units, PLL/DLL Interfaces, Frequency Dividers, Dynamic Voltage/Frequency Scaling Units, Power Gating Controllers, Wakeup Controllers, Retention Register Banks, Clock Mux/Demux Units
13	Error Handling and Fault Tolerance Modules	ECC Encoders/Decoders (SECDED, BCH, Reed-Solomon), Watchdog Timers, Parity Generators/Checkers, Redundancy Logic (TMR – Triple Modular Redundancy), Fault Injection and Detection Blocks, Error Logging Units

Acknowledgments

The author(s) would like to thank Shams Tarek for valuable assistance with the classification portion of this work.

References

- [1] Mohammad Akyash, Kimia Azar, and Hadi Kamali. 2025. RTL++: Graph-enhanced LLM for RTL Code Generation. arXiv:2505.13479 [cs.PL] <https://arxiv.org/abs/2505.13479>
- [2] Ahmed Allam and Mohamed Shalan. 2024. RTL-Repo: A Benchmark for Evaluating LLMs on Large-Scale RTL Design Projects. arXiv:2405.17378 [cs.LG] <https://arxiv.org/abs/2405.17378>
- [3] Shadan Alsaqer, Sarah Alajmi, Imtiaz Ahmad, and Mohammad Alfaiakawi. 2024. The potential of LLMs in hardware design. *Journal of Engineering Research* (2024). doi:10.1016/j.jer.2024.08.001
- [4] D.M. Anisuzzaman, Jeffrey G. Malins, Paul A. Friedman, and Zachi I. Attia. 2025. Fine-Tuning Large Language Models for Specialized Use Cases. *Mayo Clinic Proceedings: Digital Health* 3, 1 (2025), 100184. doi:10.1016/j.mcpdig.2024.11.005
- [5] Oliver Arafat and Dirk Riehle. 2009. The comment density of open source software code. In *2009 31st International Conference on Software Engineering - Companion Volume*. 195–198. doi:10.1109/ICSE-COMPANION.2009.5070980
- [6] Rudolf Bayer and Edward M McCreight. 1972. Organization and maintenance of large ordered indexes. *Acta Informatica* 1, 3 (1972), 173–189.
- [7] Jason Blocklove, Siddharth Garg, Ramesh Karri, and Hammond Pearce. 2023. Chip-Chat: Challenges and Opportunities in Conversational Hardware Design. In *Proceedings of the 2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. IEEE, 1–6. doi:10.1109/MLCAD58807.2023.10299874
- [8] Kaiyan Chang, Ying Wang, Haimeng Ren, Mengdi Wang, Shengwen Liang, Yinhe Han, Huawei Li, and Xiaowei Li. 2023. Chipgpt: How far are we from natural language hardware design. *arXiv preprint arXiv:2305.14019* (2023).
- [9] Fan Cui, Chenyang Yin, Kexing Zhou, Youwei Xiao, Guangyu Sun, Qiang Xu, Qipeng Guo, Demin Song, Dahua Lin, Xingcheng Zhang, Yun, and Liang. 2024. OriGen: Enhancing RTL Code Generation with Code-to-Code Augmentation and Self-Reflection. arXiv:2407.16237 [cs.AR] <https://arxiv.org/abs/2407.16237>
- [10] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. 2019. Class-Balanced Loss Based on Effective Number of Samples. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 9260–9269. doi:10.1109/CVPR.2019.00949
- [11] Yonggan Fu, Yongan Zhang, Zhongzhi Yu, Sixu Li, Zhifan Ye, Chaojian Li, Cheng Wan, and Yingyan Celine Lin. 2023. Gpt4aigchip: Towards next-generation ai accelerator design automation via large language models. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [12] Mingzhe Gao, Jieru Zhao, Zhe Lin, Wenchao Ding, Xiaofeng Hou, Yu Feng, Chao Li, and Minyi Guo. 2024. AutoVCoder: A Systematic Framework for Automated Verilog Code Generation using LLMs. arXiv:2407.18333 [cs.AR] <https://arxiv.org/abs/2407.18333>
- [13] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997 [cs.CL] <https://arxiv.org/abs/2312.10997>
- [14] PostgreSQL Global Development Group. [n.d.]. PostgreSQL. <https://www.postgresql.org/>
- [15] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified Cross-Modal Pre-training for Code Representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 7212–7225. doi:10.18653/v1/2022.acl-long.499
- [16] Shailja Gupta, Rajesh Ranjan, and Surya Narayan Singh. 2024. A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions. arXiv:2410.12837 [cs.CL] <https://arxiv.org/abs/2410.12837>
- [17] Guyue Huang, Jingbo Hu, Yifan He, Jialong Liu, Mingyuan Ma, Zhaoyang Shen, Juejian Wu, Yuanfan Xu, Hengrui Zhang, Kai Zhong, Xuefei Ning, Yuzhe Ma, Haoyu Yang, Bei Yu, Huazhong Yang, and Yu Wang. 2021. Machine Learning for Electronic Design Automation: A Survey. *ACM Trans. Des. Autom. Electron. Syst.* 26, 5, Article 40 (June 2021), 46 pages. doi:10.1145/3451179
- [18] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL]
- [19] Heiko Koziolok, Sten Grüner, Rhaban Hark, Virendra Ashiwal, Sofia Linsbauer, and Nafise Eskandani. 2024. LLM-based and Retrieval-Augmented Control Code Generation. In *Proceedings of the 1st International Workshop on Large Language Models for Code (Lisbon, Portugal) (LLM4Code '24)*. Association for Computing Machinery, New York, NY, USA, 22–29. doi:10.1145/3643795.3648384
- [20] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. 2022. Deduplicating Training Data Makes Language Models Better. arXiv:2107.06499 [cs.CL] <https://arxiv.org/abs/2107.06499>
- [21] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating Large Language Models for Verilog Code Generation. *arXiv preprint arXiv:2309.07544* (2023).
- [22] Mingjie Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. 2023. VerilogEval: Evaluating large language models for verilog code generation. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–8.
- [23] Mingjie Liu, Yun-Da Tsai, Wenfei Zhou, and Haoxing Ren. 2025. CraftRTL: High-quality Synthetic Data Generation for Verilog Code Models with Correct-by-Construction Non-Textual Representations and Targeted Code Repair. arXiv:2409.12993 [cs.AR] <https://arxiv.org/abs/2409.12993>

- [24] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2024. RTLCode: Fully open-source and efficient LLM-assisted RTL code generation technique. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2024).
- [25] Shang Liu, Yao Lu, Wenji Fang, Mengming Li, and Zhiyao Xie. 2025. OpenLLM-RTL: Open Dataset and Benchmark for LLM-Aided Design RTL Generation. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design* (Newark Liberty International Airport Marriott, New York, NY, USA) (ICCAD '24). Association for Computing Machinery, New York, NY, USA, Article 60, 9 pages. doi:10.1145/3676536.3697118
- [26] Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. 2024. RLLM: An Open-Source Benchmark for Design RTL Generation with Large Language Model. In *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 722–727.
- [27] Zhendong Mi, Renming Zheng, Haowen Zhong, Yue Sun, Seth Kneeland, Sayan Moitra, Ken Kutzer, and Zhaozhuo Xu Shaoyi Huang. 2025. CoopetitiveV: Leveraging LLM-powered Coopetitive Multi-Agent Prompting for High-quality Verilog Generation. arXiv:2412.11014 [cs.LG] <https://arxiv.org/abs/2412.11014>
- [28] OpenAI. 2025. Introducing Codex: An AI coding agent. <https://openai.com/index/introducing-codex> Accessed: 2025-05-17.
- [29] OpenCores Organization. 1999. OpenCores: Open-Source Hardware Community. <https://opencores.org/>
- [30] Venkatesh Balavadhani Parthasarathy, Ahtsham Zafar, Aafaq Khan, and Arsalan Shahid. 2024. The Ultimate Guide to Fine-Tuning LLMs from Basics to Breakthroughs: An Exhaustive Review of Technologies, Research, Best Practices, Applied Research Challenges and Opportunities. arXiv:2408.13296 [cs.LG] <https://arxiv.org/abs/2408.13296>
- [31] Zehua Pei, Hui-Ling Zhen, Mingxuan Yuan, Yu Huang, and Bei Yu. 2024. BetterV: Controlled Verilog Generation with Discriminative Guidance. arXiv:2402.03375 [cs.AI] <https://arxiv.org/abs/2402.03375>
- [32] Ruidi Qiu, Grace Li Zhang, Rolf Drechsler, Ulf Schlichtmann, and Bing Li. 2024. Autobench: Automatic testbench generation and evaluation using llms for hdl design. In *Proceedings of the 2024 ACM/IEEE International Symposium on Machine Learning for CAD*. 1–10.
- [33] Ronald L. Rivest. 1992. The MD5 Message-Digest Algorithm. Request for Comments (RFC) 1321. <https://tools.ietf.org/html/rfc1321>
- [34] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2024. Code Llama: Open Foundation Models for Code. arXiv:2308.12950 [cs.CL] <https://arxiv.org/abs/2308.12950>
- [35] Dipayan Saha, Hasan Al Shaikh, Shams Tarek, and Farimah Farahmandi. 2025. ThreatLens: LLM-guided Threat Modeling and Test Plan Generation for Hardware Security Verification. arXiv:2505.06821 [cs.CR] <https://arxiv.org/abs/2505.06821>
- [36] Dipayan Saha, Shams Tarek, Katayoon Yahyaei, Sujun Kumar Saha, Jingbo Zhou, Mark Tehranipoor, and Farimah Farahmandi. 2024. LLM for SoC Security: A Paradigm Shift. *IEEE Access* 12 (2024), 155498–155521. doi:10.1109/ACCESS.2024.3427369
- [37] Dipayan Saha, Katayoon Yahyaei, Sujun Kumar Saha, Mark Tehranipoor, and Farimah Farahmandi. 2024. Empowering Hardware Security with LLM: The Development of a Vulnerable Hardware Database. In *2024 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. 233–243. doi:10.1109/HOST55342.2024.10545393
- [38] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2025. A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications. arXiv:2402.07927 [cs.AI] <https://arxiv.org/abs/2402.07927>
- [39] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, Hyojung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galyuker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. 2025. The Prompt Report: A Systematic Survey of Prompt Engineering Techniques. arXiv:2406.06608 [cs.CL] <https://arxiv.org/abs/2406.06608>
- [40] Reshma Sheik, K P Siva Sundara, and S Jaya Nirmala. 2024. Neural data augmentation for legal overruling task: Small deep learning models vs. Large language models. *Neural Process. Lett.* 56, 2 (March 2024).
- [41] Jingzhe Shi, Qinwei Ma, Hongyi Liu, Hang Zhao, Jeng-Neng Hwang, Serge Belongie, and Lei Li. 2025. Explaining Context Length Scaling and Bounds for Language Models. arXiv:2502.01481 [cs.LG] <https://arxiv.org/abs/2502.01481>
- [42] Wilson Snyder and contributors. 2003–2024. Verilator: Open Source SystemVerilog Simulator. <https://verilator.org>.
- [43] Demin Song, Honglin Guo, Yunhua Zhou, Shuhao Xing, Yudong Wang, Zifan Song, Wenwei Zhang, Qipeng Guo, Hang Yan, Xipeng Qiu, and Dahua Lin. 2024. Code Needs Comments: Enhancing Code LLMs with Comment Augmentation. arXiv:2402.13013 [cs.CL] <https://arxiv.org/abs/2402.13013>
- [44] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. VeriGen: A Large Language Model for Verilog Code Generation. *ACM Transactions on Design Automation of Electronic Systems* (2024). doi:10.1145/3643681
- [45] Shailja Thakur, Baleegh Ahmad, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Ramesh Karri, and Siddharth Garg. 2024. Verigen: A large language model for verilog code generation. *ACM Transactions on Design Automation of Electronic Systems* 29, 3 (2024), 1–31.
- [46] Shailja Thakur, Jason Blocklove, Hammond Pearce, Benjamin Tan, Siddharth Garg, and Ramesh Karri. 2024. AutoChip: Automating HDL Generation Using LLM Feedback. arXiv:2311.04887 [cs.PL] <https://arxiv.org/abs/2311.04887>
- [47] Ning Wang, Bingkun Yao, Jie Zhou, Xi Wang, Zhe Jiang, and Nan Guan. 2025. Large Language Model for Verilog Generation with Code-Structure-Guided Reinforcement Learning. arXiv:2407.18271 [cs.AR] <https://arxiv.org/abs/2407.18271>
- [48] Stephen Williams. 2024. Icarus Verilog Documentation. <https://steveicarus.github.io/iverilog/>.
- [49] Clifford Wolf. 2024. Yosys Open SYnthesis Suite Documentation. <https://yosyshq.readthedocs.io/>.

- [50] Peiyang Wu, Nan Guo, Xiao Xiao, Wenming Li, Xiaochun Ye, and Dongrui Fan. 2025. ITERTL: An Iterative Framework for Fine-tuning LLMs for RTL Code Generation. arXiv:2407.12022 [cs.CL] <https://arxiv.org/abs/2407.12022>
- [51] Shangyu Wu, Ying Xiong, Yufei Cui, Haolun Wu, Can Chen, Ye Yuan, Lianming Huang, Xue Liu, Tei-Wei Kuo, Nan Guan, and Chun Jason Xue. 2025. Retrieval-Augmented Generation for Natural Language Processing: A Survey. arXiv:2407.13193 [cs.CL] <https://arxiv.org/abs/2407.13193>
- [52] Yingsheng Wu, Yuxuan Gu, Xiaocheng Feng, Weihong Zhong, Dongliang Xu, Qing Yang, Hongtao Liu, and Bing Qin. 2024. Extending Context Window of Large Language Models from a Distributional Perspective. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 7288–7301. doi:10.18653/v1/2024.emnlp-main.414
- [53] Bei Yu. 2023. Machine Learning in EDA: When and How. In *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*. 1–6. doi:10.1109/MLCAD58807.2023.10299822
- [54] Yongan Zhang, Zhongzhi Yu, Yonggan Fu, Cheng Wan, and Yingyan Celine Lin. 2024. Mg-verilog: Multi-grained dataset towards enhanced llm-assisted verilog generation. In *2024 IEEE LLM Aided Design Workshop (LAD)*. IEEE, 1–5.
- [55] Zixi Zhang, Balint Szekely, Pedro Gimenes, Greg Chadwick, Hugo McNally, Jianyi Cheng, Robert Mullins, and Yiren Zhao. 2025. LLM4DV: Using Large Language Models for Hardware Test Stimuli Generation. arXiv:2310.04535 [cs.LG] <https://arxiv.org/abs/2310.04535>
- [56] Yang Zhao, Di Huang, Chongxiao Li, Pengwei Jin, Muxin Song, Yanan Xu, Ziyuan Nan, Mingju Gao, Tianyun Ma, Lei Qi, Yansong Pan, Zhenxing Zhang, Rui Zhang, Xishan Zhang, Zidong Du, Qi Guo, and Xing Hu. 2025. CodeV: Empowering LLMs with HDL Generation through Multi-Level Summarization. arXiv:2407.10424 [cs.PL] <https://arxiv.org/abs/2407.10424>